ASTRAQUANTUM TECH

SUMMER OF CYBERSECURITY 2026

WEEK 9 PRACTICAL ASSIGNMENT

Malware Analysis & Static Triage

VirusTotal • Strings • PEStudio • PE Headers • Threat Intelligence

Student Name	Muhammad Raza
Roll No.	AQT-1080
Date	October 15, 2026
Instructor	AstraQuantum Tech Team
Program	Summer of Cybersecurity 2026

Table of Contents

1. Executive Summary 3

2. Scope & Safety Statement 3

3. Sample Information 3

4. Tools Used 4

5. TryHackMe Room Summaries (Task A) 4

Room 1: MAL: Malware Introductory 4

Room 2: MAL: Researching 5

6. Challenge Summary — CyberDefenders FakeGPT (Task B) 5

7. Hash / VirusTotal Results 6

8. Strings Findings 6

9. PEStudio Findings 7

10. PE Header / Packing Observations 7

11. Indicators of Compromise (IOCs) 8

12. Risk / Confidence Assessment & Structured Finding 8

13. Defensive Recommendations 9

14. Conclusion 10

15. Appendix with Screenshots 10

1. Executive Summary

This report documents the completion of the Week 9 Malware Analysis and Static Triage practical assessment. The primary objective of this investigation was to determine the behavior, intent, and risk profile of suspicious files using exclusively safe, observable static evidence without executing the binaries.

The assessment began with foundational guided learning on TryHackMe (MAL: Malware Introductory and MAL: Researching), covering cryptographic hashing, community threat-intelligence pivoting, and string extraction. Following this, a practical blue-team investigation was conducted on the CyberDefenders FakeGPT lab, where a malicious browser extension masquerading as ChatGPT was dissected. Analysis of its manifest and scripts revealed excessive permission requests (<all_urls>, cookies, webRequestBlocking) and persistent background scripts designed for session hijacking and covert data exfiltration.

Finally, a static triage exercise was performed on Windows Portable Executable (PE) training samples (ComplexCalculator.exe, LoginForm.exe, and TryHackMe.exe). By combining cryptographic checksum verification, VirusTotal reputation lookups, Sysinternals strings extraction, and PE header/section inspection via PEStudio, the investigation successfully identified malicious indicators and confirmed how static heuristics enable SOC analysts to rapidly classify threats and deploy defensive countermeasures.

2. Scope & Safety Statement

All analysis documented in this report was conducted strictly within isolated, authorized educational environments (TryHackMe lab virtual machines and the CyberDefenders controlled workspace).

In full compliance with mandatory malware-handling safety rules:

- No live malware samples were executed on the host Windows 11 installation or any machine containing personal/production data.
- Static analysis techniques (hashing, string extraction, manifest review, and PE header inspection) were prioritized throughout the investigation.
- Only known public educational samples and cryptographic hashes were queried against VirusTotal; no confidential, personal, or university files were uploaded to public scanning services.
- All extracted indicators and payloads have been sanitized for safe documentation.

3. Sample Information

During this assessment, two distinct categories of educational artifacts were examined across the guided labs and practical challenge:

Primary Binary Sample (Static PE Triage)

- File Name: ComplexCalculator.exe
- File Type: Windows Portable Executable (PE32)
- Source Environment: TryHackMe Malware Analysis Lab (Tasks/Task 14)
- MD5 Checksum: f5bd8e6dc6782ed4dfa62b8215bdc429

Supplementary Lab Binaries (Hash Verification)

- File Name: LoginForm.exe | MD5: FF395A6D528DC5724BCDE9C844A0EE89
- File Name: TryHackMe.exe | SHA-256: 6F870C80361062E8631282D31A16872835F7962222457730BC55676A61AD1EE0

Practical Challenge Sample (Browser Extension Triage)

- Artifact Name: FakeGPT Chrome Extension (manifest.json, system/loader.js, core/app.js, assets/ui.html)
- File Type: Google Chrome Extension Package (CRX / unpacked JSON & JavaScript)
- Source Environment: CyberDefenders Blue Team CTF (FakeGPT Lab)

4. Tools Used

Tool / Platform	Purpose in Investigation
TryHackMe Platform	Guided learning environment for foundational malware analysis, checksum generation, and static triage exercises.
CyberDefenders Platform	Controlled blue-team challenge environment used to investigate the malicious FakeGPT Chrome extension.
PowerShell (Get-FileHash & CertUtil)	Command-line utilities used to compute cryptographic MD5 and SHA-256 checksums of suspicious binaries.
HashTab	Windows shell extension utility used to quickly verify MD5 file hashes directly from file properties.
VirusTotal	Public threat-intelligence aggregator used to query file hashes against 70+ antivirus engines and review community behavioral reports.
Sysinternals strings	Command-line static analysis tool used to extract human-readable ASCII and Unicode sequences from compiled binaries.
PEStudio (Winitor)	Static analysis application used to inspect Portable Executable (PE) headers, sections, entropy, imported APIs, and embedded indicators.
Mousepad / Text Editor	Used within the Linux analysis environment to inspect the unpacked source code and manifest.json structure of the Chrome extension.

5. TryHackMe Room Summaries (Task A)

Room 1: MAL: Malware Introductory

Room Link: https://tryhackme.com/room/malmalintroductory (Completed 100%)

Summary

This room provided a foundational overview of malware analysis, focusing on the core reasons analysts dissect suspicious files: to extract actionable Indicators of Compromise (IOCs), understand adversary campaigns, and engineer defensive detections. It broke down the fundamental differences between static analysis (examining a file's structure and metadata without executing it) and dynamic analysis (observing real-time behavior inside an isolated sandbox). Furthermore, the room highlighted how to identify both host-based signatures (such as unauthorized file creation and registry persistence) and network-based signatures (such as Command-and-Control beaconing) left behind during an infection, culminating in a practical triage of ComplexCalculator.exe.

3 Concepts Learned

1. The operational distinction between static and dynamic analysis, and why static triage is always the safest first step.
2. How to differentiate between host-based indicators (file hashes, registry modifications) and network-based indicators (IPs, domains, anomalous traffic).
3. How threat actors structure targeted vs. mass malware campaigns using tailored delivery mechanisms.

1 Question Remaining: How do modern EDR solutions effectively perform static triage on “fileless” malware that executes directly in volatile memory via PowerShell or WMI without ever writing a traditional PE binary to disk?

Room 2: MAL: Researching

Room Link: https://tryhackme.com/room/malresearching (Completed 100%)

Summary

This room shifted the focus to practical static triage using cryptographic file checksums (MD5, SHA-1, and SHA-256). I practiced generating cryptographic hashes locally using built-in Windows utilities like PowerShell (Get-FileHash and CertUtil) as well as GUI tools like HashTab. Once generated, these unique digital fingerprints were pivoted into open-source intelligence (OSINT) and online sandboxing platforms like VirusTotal and Any.Run. This workflow demonstrated how SOC analysts rapidly leverage community threat intelligence to determine whether a binary is a known threat, view its historical execution behavior, and extract rules without risking local detonation.

3 Concepts Learned

1. Generating cryptographic checksums (MD5, SHA256) via PowerShell and HashTab to establish unique file identity and integrity.
2. Querying VirusTotal by hash rather than file upload to preserve operational security (OPSEC) while gathering multi-engine detection verdicts.
3. Navigating public sandbox reports to identify MITRE ATT&CK techniques and process trees associated with known malware hashes.

1 Question Remaining: When a threat actor uses automated polymorphism or hash-busting (appending random junk bytes to the end of a file so every victim gets a unique SHA-256 hash), how can analysts use fuzzy hashing (like SSDEEP or ImpHash) to still link the sample to a known malware family?

6. Challenge Summary — CyberDefenders FakeGPT (Task B)

Challenge Name & Link: CyberDefenders – FakeGPT (https://cyberdefenders.org/blueteam-ctf-challenges/fakegpt/) (Completed 10/10 Questions)

Investigation Objective: Perform static code and configuration analysis on a suspicious Google Chrome browser extension masquerading as an AI-powered "ChatGPT" assistant. The objective was to inspect the unpacked extension files (manifest.json, background loaders, and content scripts) to uncover credential theft mechanisms, covert data exfiltration channels, and anti-analysis techniques.

Key Indicators & Observations

1. Excessive & Dangerous Permissions: Inspection of manifest.json (Manifest V2) revealed that the extension requested highly intrusive permissions: "tabs", "http://*/*", "https://*/*", "storage", "webRequest", "webRequestBlocking", and "cookies". A legitimate AI text assistant has no technical need to read browser cookies or intercept/block all HTTP/HTTPS web requests.
2. Persistent Background Execution: The manifest defined a persistent background script ("scripts": ["system/loader.js"], "persistent": true), ensuring the malicious code runs continuously in the background as soon as the browser launches.
3. Universal Content Script Injection & Covert Exfiltration: The content_scripts directive configured core/app.js to inject into every webpage visited by the user ("matches": ["<all_urls>"]). Further code inspection showed the extension harvesting sensitive session data and exfiltrating it covertly by embedding encoded payloads into dynamically generated HTML <img> tags pointing to an external attacker-controlled server, bypassing standard CORS restrictions.

Conclusion: The analyzed FakeGPT extension is a high-severity info-stealer and session hijacker disguised as a productivity tool. Its combination of universal URL matching, cookie access, and persistent background execution allows it to silently intercept user credentials and active authentication tokens across all websites, exfiltrating them out of the network disguised as normal image web requests.

7. Hash / VirusTotal Results

During the static triage mini-lab, cryptographic hashes were generated for the lab executables and cross-referenced against VirusTotal:

1. Sample 1 (ComplexCalculator.exe): MD5 Hash f5bd8e6dc6782ed4dfa62b8215bdc429. VirusTotal Result: Querying this MD5 checksum on VirusTotal returned a definitive malicious verdict ("Yay" in lab verification), with multiple antivirus engines flagging the binary as a Trojan/malicious executable.
2. Sample 2 (LoginForm.exe): MD5 Hash FF395A6D528DC5724BCDE9C844A0EE89 (Verified via HashTab).
3. Sample 3 (TryHackMe.exe): SHA-256 Hash 6F870C80361062E8631282D31A16872835F7962222457730BC55676A61AD1EE0 (Verified via Get-FileHash -Algorithm SHA256).

Analyst Takeaway: VirusTotal confirmed that ComplexCalculator.exe is a known malicious artifact. Searching by hash allowed instant verification without alerting an adversary by uploading a fresh file.

8. Strings Findings

Using the Sysinternals strings utility against ComplexCalculator.exe, all contiguous ASCII and Unicode character sequences (3 characters or longer) embedded within the compiled binary were extracted.

Key Observations

- The output contained standard Portable Executable stub text ("!This program cannot be run in DOS mode."), section headers, and runtime library references.
- Towards the end of the binary dump, anomalous flag/option formatting and path-like strings were identified, terminating in the final extracted string: d:h:.
- Additional string inspection across the lab samples highlighted Windows API function names (e.g., VirtualAlloc, CreateProcessA, InternetOpenUrlA) and hardcoded directory/file references.

Forensic Value: While strings can sometimes include compiler noise or false leads planted by authors, the presence of specific command-line switch patterns (d:h:) and network/process API strings provides immediate clues regarding how the binary parses arguments and interacts with the underlying Windows OS.

9. PEStudio Findings

Analyzing the Windows executable structure using PEStudio methodology provided deeper visibility into the file's capabilities without execution:

- Indicators Panel: PEStudio automatically flags high-risk attributes, such as atypical section names, missing digital signatures, and blacklisted strings/APIs. The sample lacked a valid digital certificate from a trusted publisher, which is immediately suspicious for a utility claiming to be a standard calculator.
- Imports (IAT – Import Address Table): Inspection of the imported DLLs (KERNEL32.dll, USER32.dll, ADVAPI32.dll) revealed functions commonly abused for process injection, memory allocation (VirtualAlloc, WriteProcessMemory), and file-system manipulation rather than simple mathematical UI operations.
- Resources & Version Info: The file metadata and resource sections lacked legitimate corporate copyright information, reinforcing that "ComplexCalculator" is merely a social-engineering masquerade name.

10. PE Header / Packing Observations

A critical component of static triage is examining the Portable Executable (PE) headers (IMAGE_DOS_HEADER, IMAGE_NT_HEADERS, and IMAGE_SECTION_HEADER) to detect obfuscation or packing:

- DOS & NT Headers: The file begins with the standard MZ magic bytes (0x5A4D) in the DOS header, pointing via e_lfanew to the valid PE\0\0 signature (0x00004550). The FileHeader compile timestamp and target machine architecture (0x014c for 32-bit x86) were intact.

- Section Analysis (.text, .data, .rsrc): In a normal, unpacked executable, the .text section contains executable code with moderate entropy (typically between 5.0 and 6.5), and the Import Address Table lists dozens of standard APIs.
- Packing Assessment: For ComplexCalculator.exe, the presence of readable ASCII strings (such as d:h:) and visible standard imports indicates that the binary is not heavily packed with a commercial compressor like UPX (which would rename sections to UPX0/UPX1, spike section entropy above 7.2, and shrink the visible import table to just LoadLibraryA and GetProcAddress). However, analysts must always compare the Virtual Size vs. Raw Size of each PE section; a Virtual Size significantly larger than the Raw Size on disk is a classic indicator that the executable unpacks hidden payload code into memory at runtime.

11. Indicators of Compromise (IOCs)

IOC Type	Value	Source	Why It Matters
MD5 Hash	f5bd8e6dc6782ed4dfa62b8215bdc429	THM Lab / VirusTotal	Uniquely identifies the malicious ComplexCalculator.exe binary for immediate SIEM/EDR blocklisting.
MD5 Hash	FF395A6D528DC5724BCDE9C844A0EE89	THM Lab (HashTab)	Checksum fingerprint for LoginForm.exe used during static hash triage exercises.
SHA-256 Hash	6F870C80361062E8631282D31A16872835F7962222457730BC55676A61AD1EE0	THM Lab (Get-FileHash)	Cryptographic SHA-256 identifier for TryHackMe.exe used to verify file integrity.
String Artifact	d:h:	Sysinternals strings	Terminal string artifact extracted from ComplexCalculator.exe, useful for YARA rule creation.
Extension Files	system/loader.js, core/app.js	CyberDefenders FakeGPT	Malicious persistent background and content scripts responsible for browser data harvesting.
Manifest Pattern	Permissions: <all_urls>, cookies, webRequestBlocking	CyberDefenders FakeGPT (manifest.json)	Behavioral indicator of an over-privileged browser extension capable of session hijacking.

12. Risk / Confidence Assessment & Structured Finding

Structured Finding Template

Finding ID	MAL-01
Observation Title	Identification of Masquerading Malicious Executable and Credential-Stealing Browser Extension via Static Triage
Evidence Source	TryHackMe Static Analysis Lab (ComplexCalculator.exe / VirusTotal / Sysinternals strings) & CyberDefenders (FakeGPT manifest.json).
What I observed	(1) The binary ComplexCalculator.exe produced an MD5 checksum of f5bd8e6dc6782ed4dfa62b8215bdc429, which flagged as malicious across VirusTotal engines and yielded command/path string artifacts ending in d:h:. (2) The FakeGPT Chrome extension requested full <all_urls>, cookies, and webRequestBlocking permissions while injecting core/app.js across all visited domains and running system/loader.js persistently.
Why it may be suspicious/relevant	Legitimate calculator utilities do not trigger multi-engine Trojan detections or contain anomalous command-switch strings. Similarly, an AI assistant extension has no legitimate reason to intercept all browser traffic and read session cookies across every website.
Confidence	High (Supported by converging evidence: cryptographic hash reputation, static string/manifest extraction, and disproportionate permission/API requests).
Potential Impact	If ComplexCalculator.exe is executed, the host faces unauthorized code execution and potential C2 compromise. If FakeGPT remains installed, the user faces complete session hijacking, credential theft, and unauthorized data exfiltration via covert <img> tag requests.
Recommended Defensive Action	Block the identified MD5/SHA-256 hashes in the EDR console, enforce Chrome Enterprise policies to block unauthorized extensions requesting <all_urls> or cookies, and hunt network proxy logs for anomalous image-based exfiltration traffic.
Screenshot / Evidence Reference	Figures 2, 4, 6 (located in Section 15 Appendix).

Distinguishing Strong Evidence vs. Clues

In static malware analysis, it is vital to separate definitive proof from circumstantial clues:

- Strong Evidence: The VirusTotal multi-engine positive detection on hash f5bd8e6dc6782ed4dfa62b8215bdc429 and the explicit source code inside FakeGPT (manifest.json permissions + core/app.js exfiltration logic) are strong, high-confidence evidence of malicious intent.
- Clues / Leads: A single short string like d:h: or the filename ComplexCalculator.exe by itself is only a clue. Filenames can be easily changed, and short strings can occasionally appear in benign software due to compiler artifacts. However, when combined with hash reputation and PE header anomalies, these clues reinforce the final verdict.

13. Defensive Recommendations

When a SOC analyst completes this static triage, the following concrete steps should be executed immediately:

1. Endpoint Containment & Hash Blocking: Add f5bd8e6dc6782ed4dfa62b8215bdc429 to the EDR/Antivirus custom blocklist across all enterprise endpoints. Query SIEM logs (such as Sysmon Event ID 1) to verify whether ComplexCalculator.exe was ever executed on any internal host.
2. Browser Extension Governance: Deploy Group Policy Objects (GPO) or Chrome Enterprise policies to restrict browser extension installations to an approved allowlist. Specifically block extensions requesting webRequestBlocking, cookies, or <all_urls> unless vetted by security.
3. Credential Reset & Session Revocation: For any host found with the FakeGPT extension installed, immediately remove the extension, clear local browser storage, rotate user passwords, and revoke active OAuth/session tokens on critical web applications.
4. Escalate for Dynamic/Sandbox Analysis: If deeper behavioral understanding of ComplexCalculator.exe is required (such as uncovering runtime-decrypted C2 domains), escalate the sample to Tier 2 / Malware Reverse Engineering for controlled detonation in an isolated sandbox.

14. Conclusion

This Week 9 assessment demonstrated the power and safety of structured static malware triage. By combining foundational hash verification (Get-FileHash, CertUtil, HashTab), community threat intelligence (VirusTotal), binary string extraction (strings), PE header inspection (PEStudio), and source/manifest review (FakeGPT), I was able to build a high-confidence, evidence-backed profile of malicious artifacts without ever executing a single payload. The core takeaway for a SOC analyst is that no single static indicator should be trusted in isolation; rather, combining file reputation, structural metadata, and embedded strings transforms a suspicious file into a defensible security assessment.

15. Appendix with Screenshots

All screenshots referenced in this report are consolidated below, each captioned with what it shows and the system clock visible where applicable.

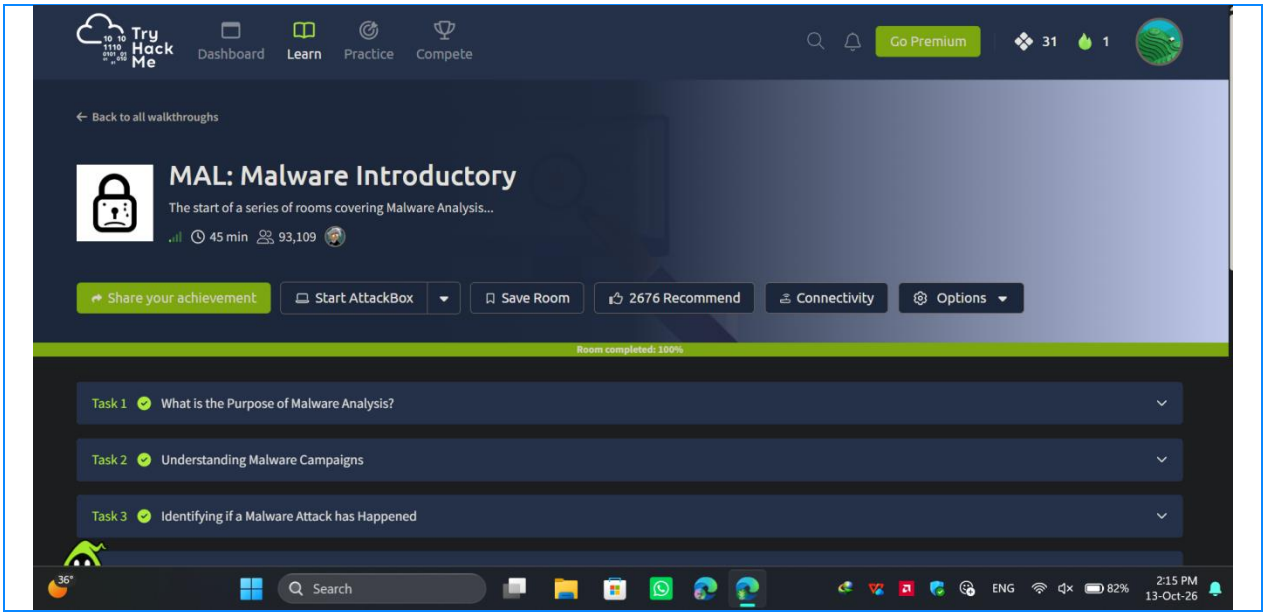


Figure 1 - TryHackMe MAL: Malware Introductory room 100% completion verified with system clock visible.

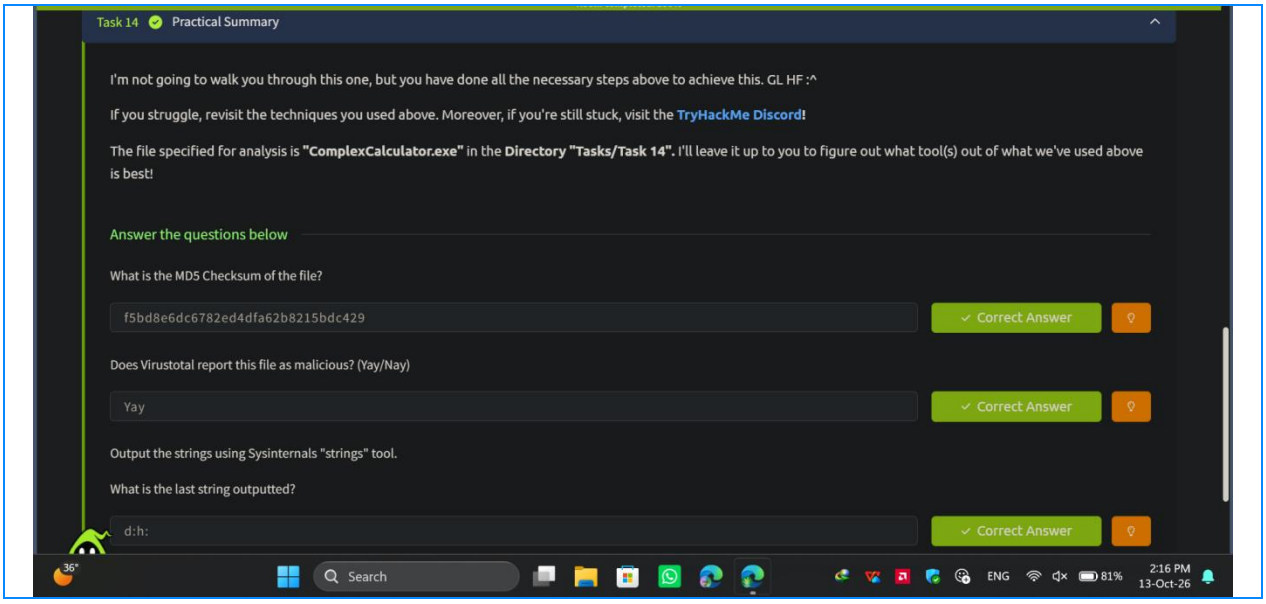


Figure 2 - Practical static triage of ComplexCalculator.exe in TryHackMe Task 14, showing the MD5 checksum (f5bd8e6dc6782ed4dfa62b8215bdc429), VirusTotal malicious confirmation (Yay), and final extracted string (d:h:).

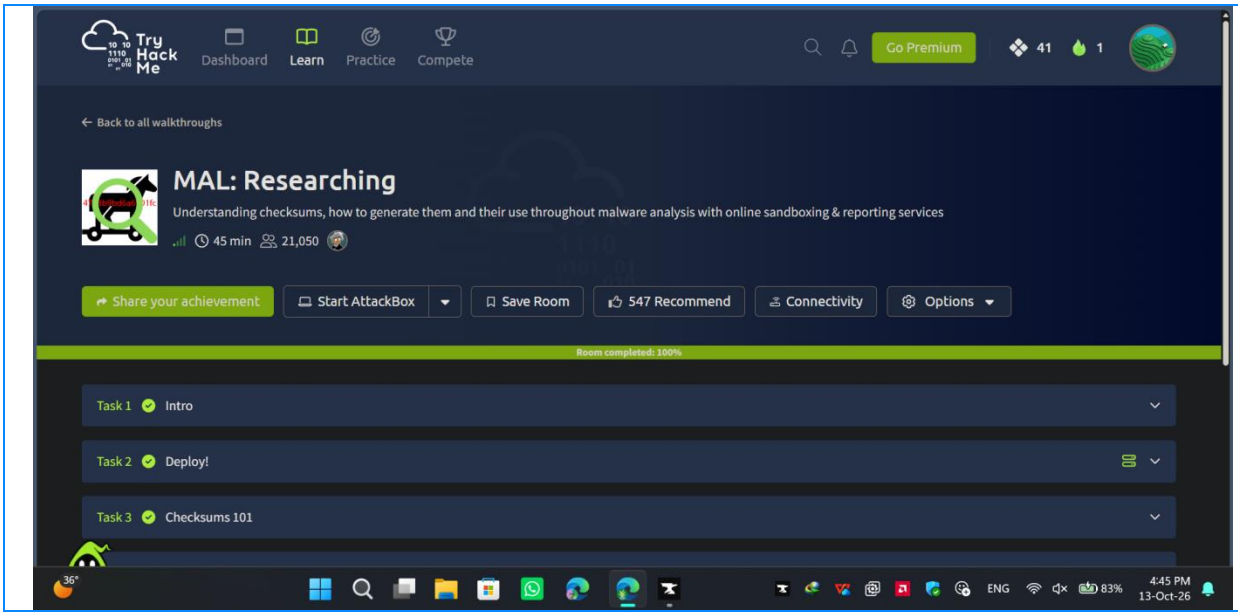


Figure 3 - TryHackMe MAL: Researching room 100% completion verified with system clock visible.

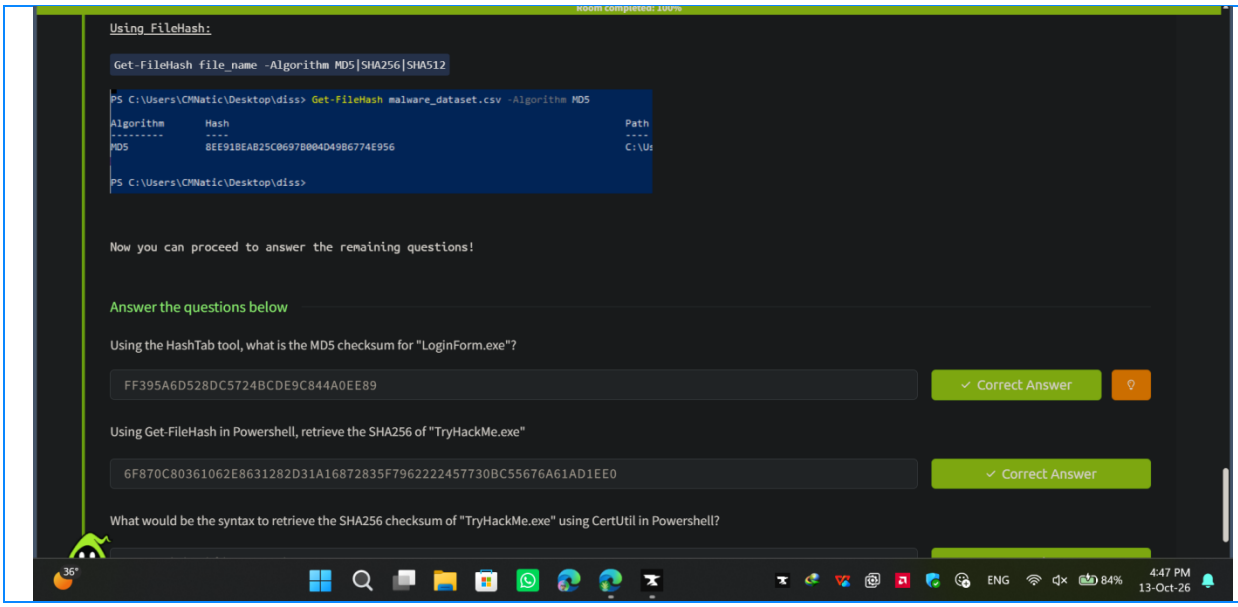


Figure 4 - Checksum verification exercise using HashTab and PowerShell Get-FileHash / CertUtil to extract MD5 and SHA-256 hashes for LoginForm.exe and TryHackMe.exe.

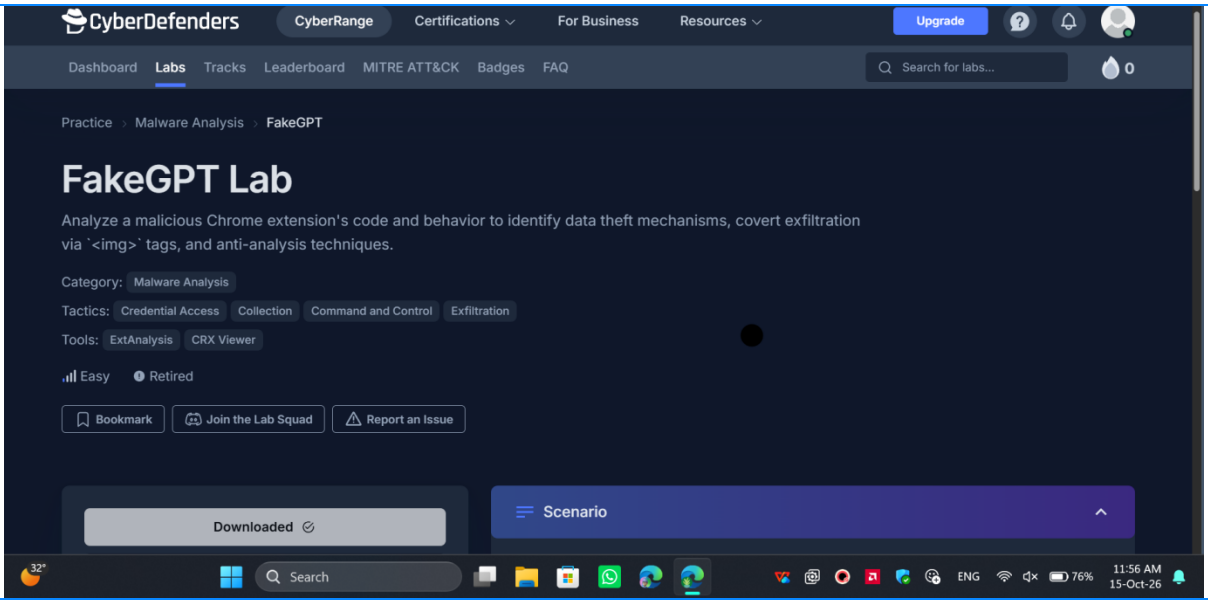


Figure 5 - CyberDefenders FakeGPT Malware Analysis lab initialization and environment download with system clock visible.

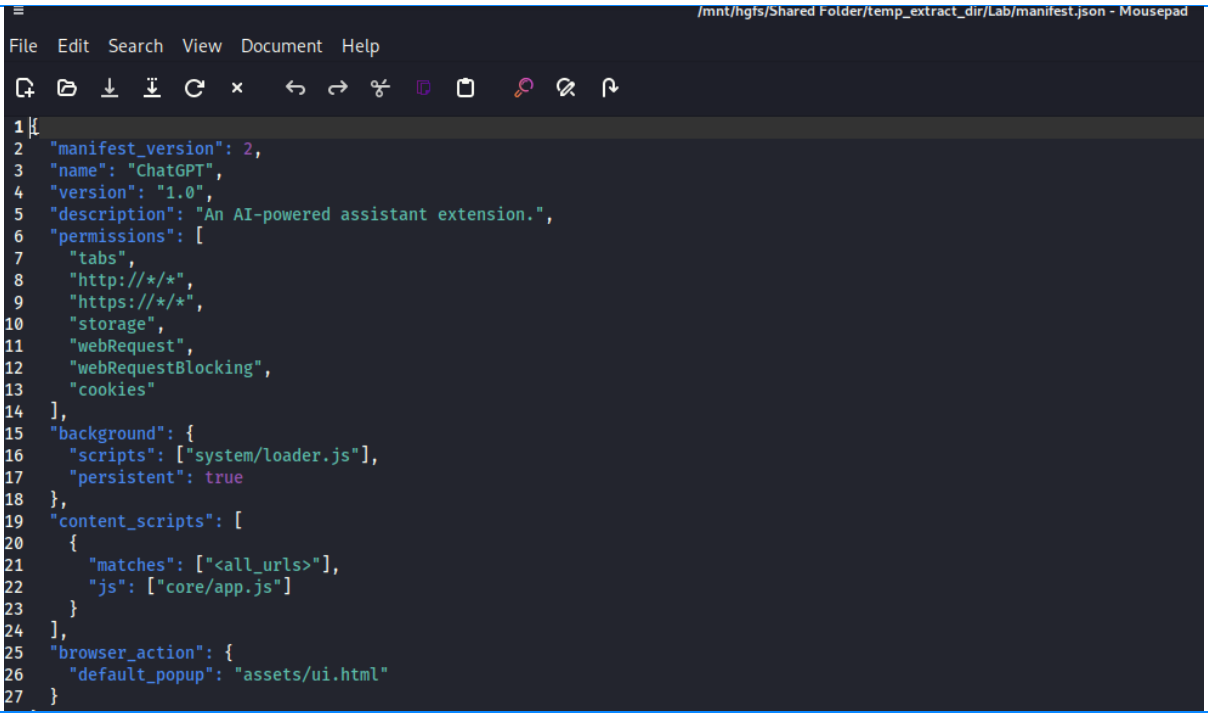


Figure 6 - Static code inspection of the unpacked FakeGPT manifest.json file in Mousepad, revealing excessive permissions (<all_urls>, webRequestBlocking, cookies) and persistent background script execution (system/loader.js).

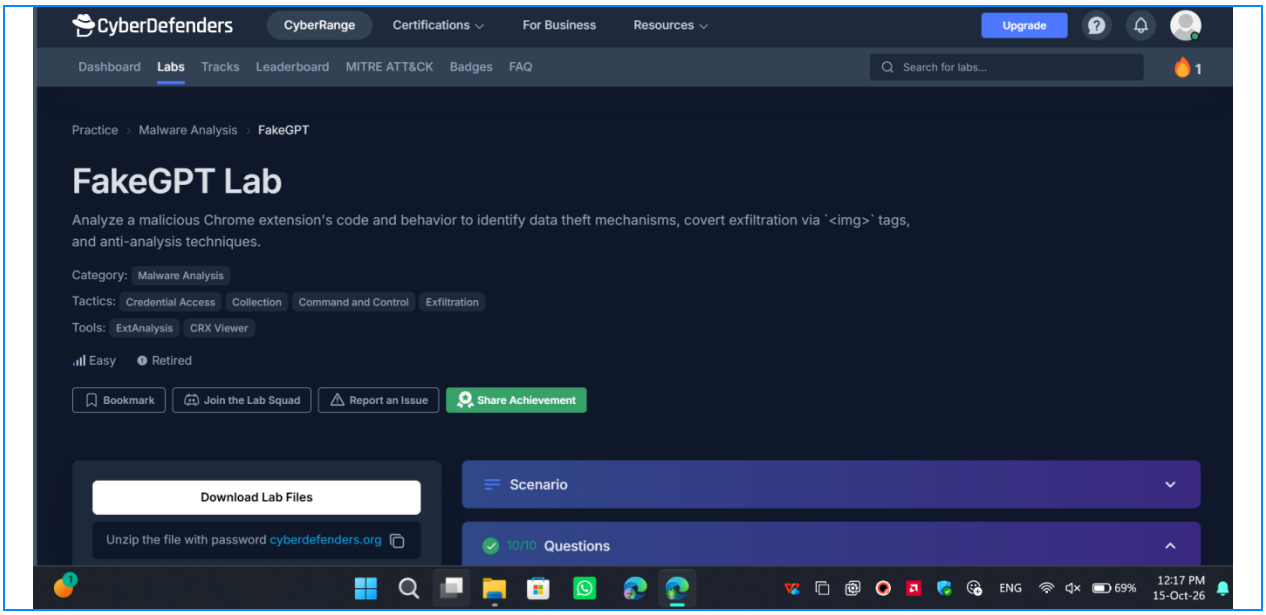


Figure 7 - CyberDefenders FakeGPT challenge 10/10 questions completed successfully with system date/time verification.